

PYTHON

03

Advanced Python Programming

Move past the basics — read and write files, tame errors, think in objects, and build real desktop apps that talk to a database.

DURATION	LEVEL	MODULE	UPDATED
11 Sessions	Intermediate	Module 3	May 2026

IN THIS MODULE

01	File Handling Basics
02	File Pointer & Cursor Control
03	Exception Handling Basics
04	Custom Exceptions
05	Classes & Objects
06	Inheritance
07	Polymorphism
08	Modules & Packages
09	Regex (re module)
10	GUI Programming with Tkinter
11	Database Connectivity with Python

SESSION

01

60 MIN · FILE I/O

File Handling Basics

◆ Learning Objectives

- Explain why programs need file handling to store data permanently beyond a single run
- Open a file using `open()` with the correct mode (`r`, `w`, `a`)
- Read file content using `read()`, `readline()`, and `readlines()`
- Write and append data to a text file using `write()`
- Close files safely using the `with` statement (context manager)

01 Why Do Programs Need Files?

Every Python program you have written so far forgets everything the moment it stops running. Variables, user inputs, calculated results — all erased. Files solve this: they store data permanently on disk so your program can read it back the next time it runs.

Imagine a medical clinic in Ahmedabad that records patient names and appointment times. Without files, every restart means starting from scratch. With file handling, that data persists automatically — no database needed for simple use cases.

CONCEPT — Persistent Storage

A file is a named collection of bytes stored on disk. Unlike a variable (which lives only in RAM while the program runs), a file survives after the program ends. Python's built-in file handling lets you create, open, read, write, and close files with just a few lines of code — no extra library required.

02 Opening a File with `open()`

`open(filename, mode)` is the entry point to any file. The first argument is the file path and the second is the mode — which tells Python what you intend to do. If you skip the mode, Python defaults to `'r'` (read-only).

Mode	Full Name	What It Does
<code>r</code>	Read	Reads an existing file. Raises an error if the file does not exist.
<code>w</code>	Write	Creates a new file or completely overwrites an existing one.
<code>a</code>	Append	Adds content to the end of a file. Creates the file if it is missing.

Mode	Full Name	What It Does
r+	Read + Write	Read and write to an existing file. Error if the file is missing.
rb / wb	Binary	Read or write raw binary data such as images or PDFs.

WARNING — 'w' Deletes All Existing Content

Opening a file with mode 'w' immediately wipes out everything already inside it — even before you write a single character. If you want to add new data without losing the old, always use 'a' (append mode) instead.

03 Reading from a File

Python gives you three reading methods. `read()` loads the entire file as one big string. `readline()` reads exactly one line and stops. `readlines()` returns a list where every element is one line — useful when you want to loop through lines later.

<code># --- Using read() ---</code>	Loads the entire file at once
<code>f = open('students.txt', 'r')</code>	Open file in read mode
<code>content = f.read()</code>	Returns the whole file as a single string
<code>print(content)</code>	Print all content
<code>f.close()</code>	Always close after use
<code># --- Using readlines() ---</code>	Loads lines as a list
<code>f = open('students.txt', 'r')</code>	Open the file again
<code>lines = f.readlines()</code>	Returns ['line1\n', 'line2\n', ...]
<code>print(lines[0])</code>	Print only the first line
<code>f.close()</code>	Close the file

TRY THIS LIVE

Create a plain text file called `students.txt` and type three student names, one per line. Run the code above and compare what `read()`, `readline()`, and `readlines()` each return. Pay attention to the difference between a single string and a list.

04 Writing and Appending to a File

`write()` sends a string directly to the file. Unlike `print()`, it does not add a newline automatically — you must include `\n` yourself at the end of each line. `writelines()` accepts a list of strings and writes them all in sequence.

<code># --- Write mode (overwrites) ---</code>	Creates or wipes the file first
<code>f = open('log.txt', 'w')</code>	Open in write mode
<code>f.write('Student: Ravi\n')</code>	Write a line — \n adds the line break
<code>f.write('Student: Priya\n')</code>	Write another line
<code>f.close()</code>	Close to flush data to disk
<code># --- Append mode (keeps old data) ---</code>	Adds to existing content
<code>f = open('log.txt', 'a')</code>	Open in append mode
<code>f.write('Student: Mehu\n')</code>	Adds to the end — does not overwrite
<code>f.close()</code>	Close when done

05 The with Statement — Close Files Automatically

Forgetting `f.close()` causes files to stay open in memory — wasting resources and sometimes corrupting data. Python's `with` statement solves this automatically: the file is closed the instant the indented block ends, even if an error occurs inside.

<code># Reading with 'with'</code>	File closes automatically when block ends
<code>with open('students.txt', 'r') as f:</code>	Open and bind file object to f
<code>content = f.read()</code>	Read inside the indented block
<code>print(content)</code>	Print it
<code># f is automatically closed here</code>	No <code>f.close()</code> call needed
<code># Writing with 'with'</code>	Same pattern works for writing
<code>with open('log.txt', 'a') as f:</code>	Open in append mode
<code>f.write('New entry\n')</code>	Append a line safely

PRO TIP — Always Use with

Every professional Python codebase uses `'with open(...) as f:'` instead of manual open/close pairs. Make this your default habit from session one — it eliminates an entire class of subtle, hard-to-trace bugs.

CASE STUDY — Batch Result Logger at a Coaching Centre

Context — A private coaching institute in Ahmedabad managing 300 students across morning and evening batches.

Challenge — Trainers were manually updating a shared spreadsheet after each test. Simultaneous edits were overwriting each other and late entries were being lost.

What they did — A Python script was written to append each student's roll number, name, and score to a shared text file immediately after each test run. The script used `open()` in `'a'` mode so no previous records were ever touched.

Result — Zero data-overwrite incidents over two months. The flat file was loaded into Excel weekly for

reporting with no manual re-entry.

Takeaway — A simple append-mode file operation solved a real coordination problem before a full database was even needed.

Source: Internal case — TOPS Technologies, Ahmedabad batch, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Search for the File I/O lecture in the later weeks of this course.

Link — <https://nptel.ac.in/courses/106106145>

Always link the official channel — beware fan-made or re-upload copies.

“A program that cannot remember anything is a program that cannot grow. Files give your code a memory.”

WHERE TO GO NEXT

You can now open, read, write, and append to text files safely using the with statement.

Explore deeper — Python official docs on open() — <https://docs.python.org/3/library/functions.html#open>

Try on your own — Build a simple contact book: ask the user for a name and phone number in a loop, append each entry to contacts.txt, then read the file back and print all saved contacts.

Connect the dots — Session 02 goes deeper — you will learn to jump to any exact position inside a file using seek() and tell(), which becomes essential when files are large.

Going further — Look up Python's pathlib module — a modern, object-oriented way to handle file paths that works identically across Windows, macOS, and Linux.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain the difference between 'r', 'w', and 'a' file modes in Python using simple Hinglish. Use a real Indian example like a school attendance register. Keep it under 150 words.

SESSION

02

60 MIN · FILE I/O

File Pointer & Cursor Control

◆ Learning Objectives

- Explain what a file pointer is and how Python tracks position inside an open file
- Use `tell()` to find the current cursor position in bytes
- Use `seek()` to jump to any byte position within a file
- Read a large file line by line using a for loop without loading it entirely into memory
- Combine `tell()` and `seek()` to re-read or skip specific sections of a file

01 What Is a File Pointer?

When Python opens a file, it places an invisible marker called the file pointer (or cursor) at a specific position inside that file. Every read or write operation moves this marker forward automatically. The next operation starts exactly where the last one left off.

Think of it like reading a novel borrowed from Ahmedabad's public library: a bookmark records exactly where you stopped. The file pointer works the same way — Python always knows where to resume without re-reading from the beginning.

CONCEPT — File Pointer

The file pointer is an integer that records the current byte offset from the start of the file. When you open a file in 'r' or 'w' mode, the pointer starts at byte 0 (the very beginning). In 'a' (append) mode, it starts at the end of the file. Every character you read or write advances this counter by the number of bytes processed.

02 `tell()` — Where Is the Cursor Right Now?

`tell()` is a method you call on an open file object. It returns an integer — the current byte position of the pointer. You pass it no arguments; it simply reports where the cursor is at that exact moment.

<code>with open('students.txt', 'r') as f:</code>	Open the file
<code> print(f.tell())</code>	Prints 0 — pointer is at the start
<code> f.read(5)</code>	Read the first 5 characters (bytes in ASCII)
<code> print(f.tell())</code>	Prints 5 — pointer moved forward by 5
<code> f.readline()</code>	Read the rest of the first line
<code> print(f.tell())</code>	Pointer is now past the end of line 1

TRY THIS LIVE

Open any text file and call `tell()` before and after each read. Watch the number grow with every read call. Try to predict the final position before running — check whether your guess was correct.

03 `seek()` — Jump to Any Position

`seek(offset, whence)` moves the file pointer to a specific byte position. The offset is how many bytes to move, and `whence` sets the reference point: 0 means from the start, 1 means from the current position, and 2 means from the end. In text mode, Python guarantees only `seek(0)` and positions previously returned by `tell()` work reliably — use binary mode for full seek flexibility.

whence value	Reference Point	Typical Use
0 (default)	Start of file	<code>seek(0)</code> — jump back to the beginning
1	Current position	<code>seek(0, 1)</code> — stay in place (limited in text mode)
2	End of file	<code>seek(0, 2)</code> — jump to the very end (binary mode)

<code>with open('students.txt', 'r') as f:</code>	Open the file
<code>content = f.read()</code>	Read all — pointer is now at the end
<code>print('End position:', f.tell())</code>	Shows end-of-file byte position
<code>f.seek(0)</code>	Move pointer back to byte 0 (start)
<code>print('After seek:', f.tell())</code>	Prints 0 — pointer reset
<code>first_line = f.readline()</code>	Read from the start again
<code>print(first_line)</code>	Prints first line as if file just opened

PRO TIP — Re-read Without Closing

`seek(0)` lets you read the same file multiple times within a single with block — no need to close and reopen it. This is faster and cleaner when you need two passes: for example, first to count all lines, then to process them row by row.

04 Reading a File Line by Line

`readlines()` loads the entire file into a list in RAM — fine for a 10 KB file, but dangerous for a large server log. The correct approach for large files is to iterate directly over the file object using a for loop. Python fetches one line at a time, discards it after processing, and keeps memory usage flat no matter how large the file is.

readlines() — All at Once

Loads the entire file into a list in RAM. Simple and convenient for small files. Risky for large files — the program can crash if the file is bigger than available memory.

for line in f — One Line at a Time

Reads one line, processes it, then discards it before fetching the next. Works safely on files of any size. This is the professional standard for production code that touches large text files.

```
# Line-by-line using a for loop (recommended)
with open('students.txt', 'r') as f:
    for line in f:
        print(line.strip())
# --- Using enumerate for line numbers ---
with open('students.txt', 'r') as f:
    for i, line in enumerate(f, start=1):
        print(i, line.strip())
```

Memory-safe for any file size

Open the file

Python fetches one line at a time

strip() removes the trailing \n

Track line index as you go

Open again

i = line number (1-based)

Print line number and content

TRY THIS LIVE

Create a file with 10 student names. Print each name with its line number using enumerate. Then use seek(0) and readline() in a loop to read only the first 3 lines without closing and reopening the file.

CASE STUDY — Processing a Large Attendance Log

Context — An IT training company in Gandhinagar maintaining a daily attendance log that grows to several thousand lines each month.

Challenge — A script using readlines() was crashing mid-month on the server when the log file exceeded the available RAM on their low-cost EC2 instance.

What they did — The script was rewritten to iterate directly over the file object (for line in f), writing each processed record to the database one row at a time. seek(0) was called at the start of each run to guarantee the pointer was always at the beginning.

Result — Memory usage dropped from around 800 MB to under 10 MB. The script now runs reliably on the smallest EC2 instance tier with no restarts.

Takeaway — Iterating line-by-line instead of loading the whole file is a mandatory habit for any production code that touches large data files.

Source: Internal infrastructure case — TOPS Technologies, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the File

Handling and I/O week in this course.

Link — <https://nptel.ac.in/courses/106106145>

“tell() tells you where you are. seek() takes you where you need to be. Together, they give you complete control over a file.”

WHERE TO GO NEXT

You can now navigate inside a file precisely using tell() and seek(), and handle files of any size safely with a for loop.

Explore deeper — Python official docs on file objects —
<https://docs.python.org/3/glossary.html#term-file-object>

Try on your own — Write a program that reads a file and records the byte position (using tell()) at the start of each line. Then ask the user for a line number and use seek() to jump directly to that line and print it — without re-reading everything from the top.

Connect the dots — Session 03 shifts from file mechanics to error handling — you will learn how to stop your program from crashing when a file is missing or the user provides unexpected input.

Going further — Look up Python's mmap module — it maps a file directly into memory and allows seek-like random access at near C-level speed, useful for very large binary files.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain what tell() and seek() do in Python file handling using a simple Hinglish analogy — like a cassette player's rewind button or a bookmark in a book. Use a real Indian example. Keep it under 150 words.

SESSION

03

60 MIN · ERROR HANDLING

Exception Handling Basics

◆ Learning Objectives

- Distinguish between a syntax error and a runtime exception
- Wrap risky code in a try-except block to prevent program crashes
- Identify and handle the most common built-in Python exceptions by name
- Use multiple except blocks to handle different error types separately
- Apply the else and finally clauses to control program flow after exception handling

01 What Is an Exception?

A syntax error is caught before the program ever runs — Python refuses to execute your file if the code is written incorrectly. An exception is different: it is an error that occurs at runtime, while the program is already running. Without handling, the program crashes and prints a traceback.

Consider a simple fee-collection program in Surat: the code is syntactically perfect, but if a staff member types 'abc' when the program expects a number, Python raises a ValueError and the whole program stops. Exception handling keeps the program alive and shows a helpful message instead.

CONCEPT — Exception vs Syntax Error

Syntax errors are writing mistakes (wrong indentation, missing colon, unclosed bracket) that Python catches before execution. Exceptions are unexpected situations that arise while the program runs — dividing by zero, opening a file that does not exist, or passing the wrong type to a function. Python represents every exception as an object from a specific built-in class: ZeroDivisionError, ValueError, FileNotFoundError, and many more.

```
# ZeroDivisionError – runs but crashes
x = 10 / 0
# ValueError – wrong value for the operation
num = int('hello')
# FileNotFoundError – file does not exist
f = open('missing_file.txt', 'r')
```

Syntactically correct, logically broken
Python raises ZeroDivisionError here
int() needs a numeric string
Raises ValueError — cannot convert 'hello'
Crash if file is absent
Raises FileNotFoundError

02 try and except — Catching Exceptions

The try block wraps code that might raise an exception. If an exception occurs inside try, Python immediately skips the rest of the try block and jumps to the matching except block. After the except block runs, the program continues normally — it does not crash.

```
try:
    num = int(input('Enter a number: '))
    result = 100 / num
    print('Result:', result)
except ValueError:
    print('Please enter a valid number.')
except ZeroDivisionError:
    print('Cannot divide by zero!')
```

Code that might fail goes inside here
Might raise ValueError
Might raise ZeroDivisionError
Only runs if both lines above succeeded
Runs if the user typed a non-number
Friendly error message
Runs if the user entered 0
Specific message for this error

TRY THIS LIVE

Run the code above three times: enter a valid number, enter 'hello', then enter 0. Watch which except block fires in each case. Notice the program does not crash — each error is caught and handled gracefully.

03 Common Python Exceptions

Python has dozens of built-in exception types. As a developer you will encounter the same small set repeatedly. Knowing their names and what triggers them lets you write precise except blocks that catch exactly the right error — and nothing more.

Exception	When It Occurs	Example That Triggers It
ValueError	Right type, but value is not acceptable	<code>int('hello')</code>
TypeError	Operation applied to the wrong data type	<code>5 + 'three'</code>
ZeroDivisionError	Division or modulo by zero	<code>10 / 0</code>
FileNotFoundError	File or directory does not exist	<code>open('x.txt', 'r')</code>
IndexError	List index is out of range	<code>my_list[100]</code>
KeyError	Dictionary key does not exist	<code>d['missing_key']</code>
NameError	Variable used before it was defined	<code>print(undefined_var)</code>

WARNING — Avoid Bare except

Writing 'except:' with no exception type catches everything — including keyboard interrupts and system exit signals. This can hide serious bugs and make programs impossible to stop cleanly. Always name the specific exception you want to catch. Use 'except Exception as e:' only as a last-resort fallback, and always print e so you can see what actually went wrong.

04 else and finally

The `else` block runs only when the `try` block completes without raising any exception — a clean way to separate success-path code from error-handling code. The `finally` block always runs, regardless of whether an exception occurred or not. Use `finally` for cleanup that must happen no matter what: closing a file, releasing a network connection, or writing a log entry.

<code>try:</code>	Code that might raise an exception
<code> num = int(input('Enter a number: '))</code>	Possible <code>ValueError</code>
<code> result = 100 / num</code>	Possible <code>ZeroDivisionError</code>
<code>except ValueError:</code>	User typed a non-number
<code> print('That was not a valid number.')</code>	Runs on <code>ValueError</code>
<code>except ZeroDivisionError:</code>	User entered 0
<code> print('Cannot divide by zero.')</code>	Runs on <code>ZeroDivisionError</code>
<code>else:</code>	Runs ONLY if try had no exception
<code> print('Result:', result)</code>	Success path — clean separation
<code>finally:</code>	ALWAYS runs — exception or not
<code> print('Calculation attempt complete.')</code>	Cleanup / audit log line

else Block

Runs only when the `try` block finishes without raising any exception. Use it to hold success-path code — this keeps the happy path clearly separated from error-handling code and makes the intent obvious to anyone reading it.

finally Block

Always runs — whether an exception was raised, caught, or never happened at all. Use it for mandatory cleanup: closing files, releasing database connections, or writing a completion record to a log file.

PRO TIP — Always Capture the Exception Object

Add `'as e'` to any `except` line to capture the exception object: `'except ValueError as e: print(e)'`. This prints the actual error message Python generated, which is invaluable for debugging and for writing meaningful entries in production log files.

CASE STUDY — Student Fee Data Processing Script

Context — A private college in Vadodara running a Python script to read student fee records from a CSV file and calculate outstanding dues.

Challenge — The script crashed whenever a fee amount cell was accidentally left blank in the CSV, because `int()` raised a `ValueError` on an empty string — halting all remaining processing.

What they did — A `try-except` block was added inside the row-processing loop. The `except` clause caught `ValueError`, logged the student's roll number and the bad row to a separate error file, and then continued to the next record using `continue`.

Result — The script now processes all clean records and generates a separate error report for rows that need manual correction — all without crashing partway through the file.

Takeaway — Exception handling inside a loop is the standard pattern for batch processing real-world data, where some records will always be malformed.

Source: Internal use case — TOPS Technologies partner institute, Vadodara, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the Exception Handling lecture in the advanced weeks of this course.

Link — <https://nptel.ac.in/courses/106106145>

“A program that crashes on bad input is not ready for real users. Exception handling is what separates a script from software.”

WHERE TO GO NEXT

You can now write try-except-else-finally blocks that handle runtime errors without crashing the program.

Explore deeper — Python official docs on built-in exceptions —
<https://docs.python.org/3/library/exceptions.html>

Try on your own — Write a program that asks the user to enter a filename. Open and read it — if the file does not exist, print a helpful message. If the file is found, print the total line count. Use finally to always print 'Attempt complete.' regardless of outcome.

Connect the dots — Session 04 builds directly on this — you will define your own custom exception classes using the raise keyword, giving your programs a professional-grade error reporting system.

Going further — Explore Python's logging module — the production-standard way to record exceptions to rotating log files with timestamps and severity levels (WARNING, ERROR, CRITICAL), instead of using bare print() statements.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain try, except, else, and finally in Python using a simple Hinglish analogy. Use a real Indian everyday example — like a UPI payment that might fail or a train ticket booking that could be rejected. Keep it under 150 words.

SESSION

04

60 MIN · ERROR HANDLING

Custom Exceptions

◆ Learning Objectives

- Explain why built-in exceptions are not specific enough for real application rules
- Use the `raise` keyword to trigger an exception deliberately
- Define a custom exception class by inheriting from the built-in `Exception` class
- Attach a meaningful message and extra data to a user-defined exception
- Read and interpret a Python traceback to pinpoint where and why an error occurred

01 When Built-in Exceptions Are Not Enough

Python's built-in exceptions cover generic situations — wrong type, missing file, index out of range. But real applications have rules that are specific to their domain: a student's age must be between 5 and 25, a fee amount cannot be negative, a course code must follow a fixed pattern.

When one of these rules is broken at an institute in Surat or Ahmedabad, raising a generic `ValueError` tells the programmer almost nothing. A custom exception like `InvalidStudentAgeError` or `NegativeFeeError` makes the problem instantly clear — in code and in log files.

CONCEPT — Custom Exceptions

A custom exception is simply a class that inherits from Python's built-in `Exception` class. Once defined, it behaves exactly like any built-in exception: you can raise it, catch it by name in an `except` block, and attach a custom error message and extra data to it. No special library is needed — just the `class` keyword.

02 The `raise` Keyword

`raise` lets you trigger an exception yourself at any point in your code. Unlike errors that happen automatically (`ZeroDivisionError` when dividing by zero), `raise` gives you deliberate control — you decide exactly when and why an exception fires, based on a business rule you define.

<code># Raising a built-in exception deliberately</code>	Enforce a business rule
<code>def set_age(age):</code>	Validate student age
<code>if age < 5 or age > 25:</code>	Rule: age must be in this range
<code>raise ValueError('Age must be between 5 and 25')</code>	
Trigger exception with a clear message	
<code>return age</code>	Return only if the rule passes
<code>try:</code>	Catch the exception
<code>set_age(30)</code>	Triggers <code>ValueError</code>

```
except ValueError as e:
    print(e)
```

Catch it
Prints: Age must be between 5 and 25

TRY THIS LIVE

Write a function `validate_fee(amount)` that raises a `ValueError` if `amount` is negative. Call it with `-500` and observe the full traceback. Then wrap the call in a try-except block to catch and print the error message without crashing.

03 Defining Your Own Exception Class

Creating a custom exception means creating a class. The class inherits from `Exception` (or from any of its built-in subclasses). You can add an `__init__` method to accept the bad value that triggered the error and build a clear, specific message from it.

Minimal custom exception — one line is enough

Inherits all behaviour from `Exception`

```
class InvalidStudentAgeError(Exception):
    pass
```

Descriptive name: what went wrong
pass — no extra behaviour needed

Custom exception with a message and data

```
class NegativeFeeError(Exception):
    def __init__(self, amount):
        self.amount = amount
```

Add context to the error
Domain-specific error for fees
Accept the bad value
Store it on the exception object

```
super().__init__(f'Fee cannot be negative: {amount}')
```

Pass message to `Exception` base

Raising and catching the custom exception

```
def set_fee(amount):
    if amount < 0:
        raise NegativeFeeError(amount)

try:
    set_fee(-1000)
except NegativeFeeError as e:
    print(e)
```

Works just like built-ins
Validation function
Rule check
Raise with the bad value
Triggers `NegativeFeeError`
Catch only this exception type
Prints: Fee cannot be negative: -1000

PRO TIP — Name Exceptions Like Events

A good custom exception name describes what went wrong as an event: `InvalidStudentAgeError`, `CourseCapacityExceededError`, `DuplicateEnrollmentError`. Avoid vague names like `MyError` or `AppException`. Clear, specific names mean you can search your log files and understand a problem without needing a

developer to investigate.

04 Reading a Python Traceback

A traceback is Python's crash report. It shows the exact chain of function calls that led to the error, and marks the file name, line number, and the specific exception that was raised. The fastest way to read it is bottom to top — the last line is always the most important.

Traceback (most recent call last):	Start of the crash report
File 'main.py', line 15, in <module>	Outermost call — where execution started
set_fee(-1000)	The line that triggered the chain
File 'main.py', line 8, in set_fee	Inside the function where raise was called
raise NegativeFeeError(amount)	The exact raise statement
NegativeFeeError: Fee cannot be negative: -1000	
READ THIS FIRST — type + message = the answer	

CONCEPT — How to Read a Traceback

Always start at the very bottom line — that is the exception type and message, which tells you what went wrong. Then read upwards to see the call stack: the path your code took to reach the crash. The line directly above the exception is almost always where the fix needs to go.

CASE STUDY — Student Enrollment Validator

Context — A college in Vadodara building a Python-based enrollment system for their diploma and certificate programs.

Challenge — When students submitted forms with invalid ages or non-existent course codes, the system raised a generic `ValueError` whose message gave the support team no actionable information — every error looked the same in the logs.

What they did — Three custom exception classes were defined: `InvalidAgeError`, `UnknownCourseCodeError`, and `DuplicateEnrollmentError`. Each stored the bad input value and a clear human-readable message. Every except block logged the specific exception class name alongside the student ID.

Result — Support ticket resolution time dropped significantly because staff could read the log entry and understand the problem without escalating to a developer.

Takeaway — Custom exceptions are not just a coding technique — they are a communication tool between the developer and everyone who reads the logs.

Source: Internal case — TOPS Technologies partner institute, Vadodara, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the exceptions and error handling week.

Link — <https://nptel.ac.in/courses/106106145>

“A custom exception is a message from the developer to the future — it says exactly what went wrong, even when no developer is in the room.”

WHERE TO GO NEXT

You can now define custom exception classes, raise them deliberately, and read any Python traceback confidently.

Explore deeper — Python official docs on user-defined exceptions — <https://docs.python.org/3/tutorial/errors.html#user-defined-exceptions>

Try on your own — Build a validator for a fictional Ahmedabad institute: define one custom exception each for invalid age, negative fee, and unknown course code. Test every rule with bad values and confirm each exception prints a clear, specific message.

Connect the dots — Session 05 shifts completely from error handling to object-oriented programming — you will define your first class, write an `__init__` method, and create objects that model real-world entities like students and courses.

Going further — Look up Python exception chaining: `'raise NewError() from original_error'`. This is the professional way to wrap a low-level exception inside a domain-specific one without losing the original traceback.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python debugger assistant. I have this Python traceback: [paste your traceback here]. Explain line by line what went wrong, in simple Hinglish. Tell me which exact line to fix and why. Keep it under 200 words and avoid technical jargon.

SESSION

05

60 MIN · OBJECT-ORIENTED PROGRAMMING

Classes & Objects

◆ Learning Objectives

- Explain the difference between a class (blueprint) and an object (instance)
- Define a class using the class keyword with correct PascalCase naming
- Initialize object attributes using the `__init__` method and the self parameter
- Create multiple objects from the same class, each holding different data
- Define instance methods and call them on individual objects

01 What Is a Class? What Is an Object?

A class is a blueprint. An object is a real thing built from that blueprint. Just as a single house plan can be used to construct many houses — each with different owners and paint colours — a single class can be used to create many objects, each holding its own unique data.

Think of a student admission form at a college in Ahmedabad. The form layout is the class: it defines the fields every student record needs — name, roll number, course. Each filled-in form for a specific student is an object: same structure, different data.

CONCEPT — Class vs Object

A class defines the structure and behaviour that all objects of its type will share. An object (also called an instance) is one concrete example of that class, holding its own copy of the data. You define a class once and create as many objects from it as you need — each independent of the others.

02 Defining a Class

Use the class keyword followed by the class name and a colon. Class names use PascalCase — each word starts with a capital letter, no underscores. Everything inside the class body is indented one level, just like a function body.

Simplest valid class — no attributes or methods yet

Just the structure

```
class Student:
    pass
```

class keyword + PascalCase name

pass — placeholder until we add content

Creating an object from the class

```
s1 = Student()
s2 = Student()
print(type(s1))
```

Call the class like a function

s1 is now an object (instance) of Student

s2 is a different object — same blueprint

Prints: <class '__main__.Student'>

```
print(s1 is s2)
```

Prints: False — two separate objects

TRY THIS LIVE

Define an empty class called Course. Create two objects c1 and c2 from it. Print type(c1) and then check c1 is c2. Confirm they are separate objects from the same blueprint even though neither holds any data yet.

03 The `__init__` Method and self

`__init__` is Python's constructor — a special method that runs automatically every time a new object is created from the class. You use it to assign starting values (called attributes) to that specific object. `self` is always the first parameter; it refers to the exact object being created.

<code>class Student:</code>	Define the class
<code>def __init__(self, name, roll_no, course):</code>	
Runs on every new object	
<code>self.name = name</code>	Attach name to this specific object
<code>self.roll_no = roll_no</code>	Attach roll number
<code>self.course = course</code>	Attach course name
<code># Creating objects – arguments go to __init__ automatically</code>	
<code>s1 = Student('Ravi', 101, 'Python')</code>	First object with its own data
<code>s2 = Student('Priya', 102, 'Data Science')</code>	Second object — completely independent
<code>print(s1.name)</code>	Prints: Ravi
<code>print(s2.course)</code>	Prints: Data Science
<code>print(s1.roll_no, s2.roll_no)</code>	Prints: 101 102

CONCEPT — What Is self?

`self` is a convention, not a keyword. It refers to the specific object on which the method is being called. When you write `s1.name`, Python internally translates that to `Student.name(s1)` — `self` tells the method which object's data to access. Always include it as the first parameter of every method, even though you never pass it yourself when calling.

Component	What It Is	Example
class	Keyword to define a class	class Student:
<code>__init__</code>	Constructor — runs on every new object	def <code>__init__</code> (self, name):

Component	What It Is	Example
self	Reference to the current object	self.name = name
Attribute	Variable attached to a specific object	s1.name, s1.course
Instance	A single object created from the class	s1 = Student('Ravi', ...)

04 Adding Methods to a Class

A method is a function defined inside a class. Like `__init__`, every method takes `self` as its first parameter. Methods give objects behaviour — things they can do — not just data they hold. You call a method on an object using dot notation: `object.method()`.

```
class Student:
    def __init__(self, name, roll_no, course):
        self.name = name
        self.roll_no = roll_no
        self.course = course

    def display(self):
        print(f'Name: {self.name} | Roll: {self.roll_no} | Course: {self.course}')

    def greet(self):
        return f'Hi, I am {self.name} enrolled in {self.course}.'
```

Full class with attributes and two methods

Constructor

Method — prints student details

Method — returns a greeting string

Uses `self` to access this object's data

Returns instead of printing

Using the methods

Call on specific objects using dot notation

Create an object

Name: Ravi | Roll: 101 | Course: Python

Hi, I am Ravi enrolled in Python.

PRO TIP — Define `__str__` for Readable Printing

If you define a `__str__` method that returns a string, Python uses it automatically whenever you call `print()` on that object. Without it, `print(s1)` produces an unreadable memory address like `<__main__.Student object at 0x...>`. Add: `def __str__(self): return f'Student: {self.name} ({self.course})'` and `print(s1)` will now show something useful.

CASE STUDY — Student Record System at a Training Institute

Context — A Python batch at TOPS Technologies, Ahmedabad, building a mini student management system as their capstone classroom project.

Challenge — Students were storing all data in separate lists and dictionaries. As the project grew, related data was scattered across multiple variables with no structure — making the code hard to read, extend, and debug.

What they did — A Student class was introduced with attributes for name, roll number, course, and batch. Methods were added for `display()`, `calculate_fee()`, and `generate_id()`. Each student became one object — data and behaviour in one place.

Result — The codebase shrank from 150 lines of tangled variable management to 60 lines of clean, class-based code. Adding a new attribute required changing only the `__init__` method — not scattered updates across 10 different lists.

Takeaway — A class bundles related data and behaviour together, making code easier to read, extend, and hand off to another developer.

Source: Classroom project — TOPS Technologies, Ahmedabad Python batch, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the Object-Oriented Programming introduction week.

Link — <https://nptel.ac.in/courses/106106145>

“A class is just a blueprint. The moment you create an object, the blueprint comes to life — with its own identity and its own data.”

WHERE TO GO NEXT

You can now define a class with `__init__` and instance methods, and create multiple independent objects from the same blueprint.

Explore deeper — Python official docs on classes — <https://docs.python.org/3/tutorial/classes.html>

Try on your own — Model a Doctor class with attributes: name, specialization, hospital, and `fee_per_consultation`. Add a method `describe()` that prints all the doctor's details using an f-string. Create three doctor objects with real specializations and call `describe()` on each.

Connect the dots — Session 06 introduces inheritance — how a child class automatically gets all the attributes and methods of a parent class, eliminating repetition when modeling related entities like Student, Staff, and Admin.

Going further — Look up Python `@classmethod` and `@staticmethod` decorators. They let you define methods that operate on the class itself rather than on individual objects — useful for factory methods and utility functions tied to a class.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain the difference between a class and an object in Python using a simple Hinglish analogy — like an Aadhaar card form versus a specific person's filled-in card, or a recipe versus the dish made from it. Keep it under 150 words.

SESSION

06

60 MIN · OBJECT-ORIENTED PROGRAMMING

Inheritance

◆ Learning Objectives

- Explain what inheritance is and why it eliminates code duplication across related classes
- Create a child class that automatically inherits attributes and methods from a parent class
- Override a parent method in a child class to change its behaviour
- Use `super()` to call the parent class constructor from a child class `__init__`
- Build a multilevel inheritance chain spanning three levels
- Define a class with two parent classes (multiple inheritance) and explain Method Resolution Order

01 What Is Inheritance?

Inheritance lets one class take on all the attributes and methods of another class automatically. The existing class is called the parent (or base class). The new class is called the child (or derived class). The child gets everything the parent has, and can add its own new features on top.

Think of a family in Gujarat: a child inherits the family name, the home address, and certain habits from the parents. In the same way, a child class inherits the structure of its parent class — and can then add or override things specific to itself.

CONCEPT — Why Inheritance Matters

Without inheritance, you would copy the same `__init__` code and the same methods into every related class — Student, Teacher, Admin — even though 80% of the code is identical. Inheritance solves this: define the shared part once in a parent class, then let each child class inherit it and only write what is unique to itself.

02 Single Inheritance

Single inheritance means one child class inherits from exactly one parent class. To create a child class, write the parent class name inside parentheses after the child class name. The child automatically has access to every method and attribute of the parent.

```
# Parent class                                Shared structure for all people
class Person:
    def __init__(self, name, age):              Common attributes
        self.name = name
        self.age = age
    def introduce(self):                        Shared method
print(f'Hi, I am {self.name}, aged {self.age}.')
```

```
# Child class inheriting from Person
class Student(Person):
    def __init__(self, name, age, course):
        super().__init__(name, age)
        self.course = course
    def study(self):
        print(f'{self.name} is studying {self.course}.')
```

Inherits name, age, introduce()
 Person in parentheses = inherit from Person
 Child adds its own attribute
 Call parent `__init__` — avoids duplicating code
 Student-specific attribute
 Student-specific method

```
s = Student('Ravi', 20, 'Python')
s.introduce()
s.study()
```

Create a student object
 Inherited from Person — Hi, I am Ravi, aged 20.
 Own method — Ravi is studying Python.

CONCEPT — `super().__init__()`

`super()` gives you a reference to the parent class. Calling `super().__init__(name, age)` inside the child's `__init__` runs the parent's constructor, setting up the parent's attributes (name, age) on the child object. Without this call, the child object would not have the parent's attributes — even though it inherits the parent's methods.

TRY THIS LIVE

Add a Trainer class that inherits from Person and adds a subject attribute and a `teach()` method. Create one Trainer object and call both `introduce()` (inherited) and `teach()` (its own). Confirm you did not duplicate the name or age code.

◆ Method Overriding

A child class can redefine a method that it inherited from the parent. The child's version replaces the parent's version for objects of the child type. This is called method overriding — the child keeps the same method name but changes what it does.

```
class Staff(Person):
    def __init__(self, name, age, department):
        super().__init__(name, age)
        self.department = department
    def introduce(self):
        print(f'I am {self.name}, staff in {self.department} dept.')
```

Another child of Person
 Override the parent's introduce()
 Different message — same method name

```
p = Person('Priya', 30)
t = Staff('Mehul', 35, 'IT')
p.introduce()
```

Parent object
 Child object
 Hi, I am Priya, aged 30.


```
t.introduce()
```

I am Mehul, staff in IT dept. (overridden)

03 Multilevel Inheritance

Multilevel inheritance is a chain: Class B inherits from Class A, and Class C inherits from Class B. Class C automatically gets everything from both A and B. There is no limit to how many levels deep a chain can go, but more than three or four levels usually makes code harder to follow.

```
# Level 1 — Grandparent
class Grandparent:
    def family_name(self):
        print('Family: Shah')

# Level 2 — Parent inherits from Grandparent
class Parent(Grandparent):
    def city(self):
        print('City: Ahmedabad')

# Level 3 — Child inherits from Parent (and indirectly from Grandparent)
class Child(Parent):
    def hobby(self):
        print('Hobby: Coding')

c = Child()
c.family_name()
c.city()
c.hobby()
```

Family example: top of the chain

Child object

From Grandparent — Family: Shah

From Parent — City: Ahmedabad

Own method — Hobby: Coding

04 Multiple Inheritance

Multiple inheritance means one child class inherits from two or more parent classes. List all parent classes inside the parentheses, separated by commas. The child gets all methods from all parents. If both parents define a method with the same name, Python uses the Method Resolution Order (MRO) to decide which one runs — left to right as listed.

```
class Student:
    def study(self):
        print('Studying hard.')

class Mentor:
    def guide(self):
```

First parent

Second parent

```
print('Guiding juniors.')
```

```
# TeachingAssistant inherits from both
class TeachingAssistant(Student, Mentor):
    def assist(self):
        print('Assisting in lab.')
```

Multiple inheritance
Both in parentheses
Own additional method

```
ta = TeachingAssistant()
ta.study()
ta.guide()
ta.assist()
print(TeachingAssistant.__mro__)
```

From Student — Studying hard.
From Mentor — Guiding juniors.
Own method — Assisting in lab.
Shows MRO: TA > Student > Mentor > object

Single Inheritance

One child, one parent. Simple chain. Use when one class is a more specific version of another — Student is a more specific Person. This covers 90% of real-world use cases.

Multiple Inheritance

One child, two or more parents. Use when a class genuinely needs capabilities from two unrelated sources — like a TeachingAssistant who is both a Student and a Mentor. Use sparingly to avoid confusion.

WARNING — Use Multiple Inheritance Carefully

Multiple inheritance becomes hard to maintain when both parent classes define the same method name. Python resolves this using the MRO (left to right), but it can surprise you. If you find yourself using multiple inheritance often, consider using composition (one class containing another as an attribute) instead.

CASE STUDY — Institute Staff Hierarchy Model

Context — A TOPS Technologies training centre in Rajkot building an internal staff management module as part of a Python capstone project.

Challenge — The team had separate, nearly-identical classes for Student, Trainer, Admin, and Placement Officer — all with duplicate code for name, age, email, and a display() method that was copy-pasted across all four files.

What they did — A Person base class was defined with the common attributes and a shared display() method. Student, Trainer, and Admin each inherited from Person using super().__init__(). Each child class only defined what was unique to its role.

Result — Code duplication dropped by 60%. When a new contact_number field was needed, the team added it once to Person.__init__ and all four derived classes automatically gained the attribute.

Takeaway — Inheritance makes shared structure the single source of truth — one change propagates everywhere it is needed.

Source: Capstone project — TOPS Technologies, Rajkot Python batch, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the OOP: Inheritance and Polymorphism lecture week.

Link — <https://nptel.ac.in/courses/106106145>

“Inheritance is how a child class says: I am everything my parent is — and more.”

WHERE TO GO NEXT

You can now build single, multilevel, and multiple inheritance chains, override parent methods, and use `super()` correctly.

Explore deeper — Python official docs on inheritance — <https://docs.python.org/3/tutorial/classes.html#inheritance>

Try on your own — Build a three-level chain: Vehicle -> Car -> ElectricCar. Vehicle has `start()`. Car overrides `start()` with an engine sound. ElectricCar overrides it with a silent motor message. Create one object of each and call `start()` — observe how each level responds differently.

Connect the dots — Session 07 covers polymorphism — how different classes can share the same method name but each produce a different result, and how Python handles this naturally without extra configuration.

Going further — Look up the Mixin pattern in Python — a common design approach where small, focused classes (not full parent classes) are combined through multiple inheritance to add specific capabilities like logging or serialization to any class.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain single, multilevel, and multiple inheritance in Python using a simple Hinglish analogy. Use a real Indian family or workplace example — like a joint family inheriting property, or an employee who reports to two managers. Keep it under 200 words.

SESSION

07

60 MIN · OBJECT-ORIENTED PROGRAMMING

Polymorphism

◆ Learning Objectives

- Define polymorphism and explain how it makes code flexible and extensible
- Apply method overriding to give child classes their own version of a parent method
- Write a single loop that calls the same method on objects of different classes
- Simulate method overloading in Python using default arguments and *args
- Explain duck typing and how Python uses it to achieve polymorphic behaviour naturally

01 What Is Polymorphism?

Polymorphism means one name, many forms. In Python, it means you can call the same method name on objects of different classes and each object responds in its own way — without the calling code needing to know which class it is dealing with.

Think of how the word 'pay' means something different at a petrol pump in Ahmedabad (you swipe a card), at a vegetable market (you hand over cash), and on Swiggy (you tap UPI). The action is the same word — but each context executes it differently.

CONCEPT — Polymorphism

Polymorphism comes from Greek: poly (many) + morph (form). In OOP, it means a single method name can produce different behaviour depending on which object it is called on. It is what allows you to write one loop that processes many different object types — each doing the right thing for itself.

02 Method Overriding — Same Name, Different Behaviour

Method overriding is the most common form of polymorphism in Python. A child class defines its own version of a method inherited from the parent. When you call that method, Python uses the child's version for child objects and the parent's version for parent objects — automatically.

# Parent class — generic payment	Base behaviour
class Payment:	
def process(self, amount):	Generic method
print(f'Processing Rs {amount}.')	
 # Child classes — each overrides process()	Different behaviour, same name
class UPIPayment(Payment):	
def process(self, amount):	Override

```
print(f'Sending Rs {amount} via GPay / PhonePe.')

class CashPayment(Payment):
    def process(self, amount):
        print(f'Collecting Rs {amount} cash at counter.')

class CardPayment(Payment):
    def process(self, amount):

# Polymorphism in action – one loop, three behaviours
payments = [UPIPayment(), CashPayment(), CardPayment()]
    Mixed list of objects
for p in payments:
    p.process(5000)
```

TRY THIS LIVE

Add a WalletPayment class (for Paytm / Amazon Pay) that overrides process() with its own message. Add an instance to the payments list and run the loop again — the new class fits in without changing a single line of the loop.

03 Method Overloading in Python (Conceptual)

In languages like Java and C++, you can define the same method name multiple times with different parameter counts — the compiler picks the right version. Python does not support this natively: if you define the same method name twice, the second definition simply replaces the first.

Python achieves the same flexibility in two cleaner ways: default parameter values for optional arguments, and *args to accept any number of arguments. Both let one function handle multiple calling patterns without duplicating the method name.

```
# Approach 1 – Default parameter values
def greet(name, message='Namaste'):
    print(f'{message}, {name}!')
```

One definition, two calling styles
message is optional — defaults to 'Namaste'

```
greet('Ravi')
greet('Priya', 'Good morning')
```

Namaste, Ravi! (uses default)
Good morning, Priya! (custom message)

```
# Approach 2 – *args for any number of arguments
def add(*numbers):
```

Handles 2, 3, or 10 values
*numbers collects all args as a tuple

Method Overriding

Child class replaces a parent method with its own version. Requires inheritance. Python picks the right version based on which class the object belongs to. Used when different classes share a common parent and a related but distinct behaviour.

Duck Typing

Unrelated classes all define the same method name independently — no inheritance needed. Python calls whichever version belongs to the object. Used when you want a common interface across classes that do not share a parent.

PRO TIP — isinstance() for Safe Type Checks

Duck typing is powerful but can cause confusing errors if an object does not have the expected method. Use `isinstance(obj, ClassName)` to check the type before calling, or use `hasattr(obj, 'method_name')` to check whether a specific method exists — without fully abandoning Python's flexibility.

CASE STUDY — Notification System at an Ed-Tech Platform

Context — An ed-tech company in Ahmedabad sending post-class notifications to students across three channels: WhatsApp, Email, and SMS.

Challenge — The notification logic was scattered across three separate if-else branches in the main code. Adding a new channel required modifying the core dispatch function every time.

What they did — Three classes were created: WhatsAppNotifier, EmailNotifier, SMSNotifier — each with a `send(message)` method. The dispatch function accepted any notifier object and called `send()` on it. Polymorphism handled the rest.

Result — Adding a new PushNotifier took five minutes: define the class, implement `send()`, add it to the notifier list. The dispatch function required zero changes.

Takeaway — Polymorphism makes systems open for extension but closed for modification — new behaviour without touching existing code.

Source: Internal architecture case — TOPS Technologies platform team, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the Polymorphism and OOP advanced concepts week.

Link — <https://nptel.ac.in/courses/106106145>

“Polymorphism lets you write code that talks to many different objects using one common language — without knowing or caring which object is on the other end.”

WHERE TO GO NEXT

You can now apply method overriding, simulate overloading, and use duck typing to write flexible, loop-driven code that handles multiple object types.

Explore deeper — Python official docs on special methods —

<https://docs.python.org/3/reference/datamodel.html#special-method-names>

Try on your own — Build a Shape system with three classes: Circle, Rectangle, Triangle. Each has an `area()` method that calculates and returns the correct area. Put all three in a list and loop through it, printing each area — one loop, three formulas.

Connect the dots — Session 08 steps outside of OOP and covers Modules and Packages — how to organize your Python code across multiple files so that classes, functions, and constants can be reused across different projects.

Going further — Look up Python's Abstract Base Classes (ABC) module. It lets you define a formal interface — a parent class with abstract methods that every child class is forced to implement, making the polymorphism contract explicit and enforced.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain polymorphism, method overriding, and duck typing in simple Hinglish. Use a real Indian example — like the same UPI, cash, or card payment methods all called 'pay' but doing different things. Keep it under 200 words.

SESSION

08

60 MIN · CODE ORGANISATION

Modules & Packages

◆ Learning Objectives

- Explain what a module is and why splitting code into files improves maintainability
- Import a module using four different styles — import, from...import, alias, and selective
- Use the math module for common mathematical operations
- Use the random module to generate random numbers and make random selections
- Create a custom module and organise multiple modules into a package with `__init__.py`

01 What Is a Module?

A module is simply a Python file (.py) that contains functions, classes, or variables. Instead of putting all your code into one large file, you split it into logical units — each module handles one responsibility. Any other Python file can then import and reuse what that module defines.

Think of how a large textile factory in Surat divides into separate departments — cutting, stitching, finishing, packing — each doing one job well. Python modules work the same way: one file per responsibility, imported wherever needed.

CONCEPT — Module vs Script

A script is a Python file you run directly. A module is a Python file you import into another file. The same file can be both: when run directly it behaves as a script, and when imported it behaves as a module. Python's `__name__` variable equals `'__main__'` when the file is run directly, and equals the module name when it is imported.

02 Four Ways to Import a Module

Python gives you four import styles. Each has a different trade-off between convenience and clarity. Choosing the right one depends on how many things you need from the module and how often you will type its name.

```
# Style 1 — import the whole module
import math
print(math.sqrt(144))
```

Access with `module.function()`

12.0 — module name is always visible

```
# Style 2 — import a specific item
from math import sqrt
print(sqrt(144))
```

Use the function directly, no prefix

Only `sqrt` is imported

12.0 — cleaner, but origin is hidden

Style 3 — import with an alias

```
import math as m
```

```
print(m.sqrt(144))
```

Short name for a long module name

Alias 'm' replaces 'math'

12.0 — shorter prefix

Style 4 — import multiple specific items

```
from math import sqrt, ceil, floor, pi
```

```
print(pi)
```

Pick exactly what you need

Four things imported at once

3.141592653589793

Style	Syntax	When to Use
Full import	<code>import math</code>	When you use many things from the module
Specific import	<code>from math import sqrt</code>	When you only need one or two items
Alias	<code>import math as m</code>	Long module names used very frequently
Multi-specific	<code>from math import sqrt, pi</code>	Multiple items, no name prefix needed
Avoid this	<code>from math import *</code>	Imports everything — pollutes namespace, avoid

PRO TIP — Never Use `import *`

`from module import *` imports every name from the module into your file. This can silently overwrite your own variable names and makes it impossible to tell where a function came from. Always import explicitly — either the module or specific names from it.

03 The math and random Modules

`math` is a built-in Python module for mathematical operations: square roots, ceiling, floor, powers, trigonometry, and constants like `pi` and `e`. It operates on floats and returns floats. `random` is another built-in module that generates pseudo-random numbers — useful for simulations, OTP generation, shuffling, and sampling.

```
import math
```

```
print(math.sqrt(81))
```

9.0 — square root

```
print(math.ceil(4.2))
```

5 — round up to nearest integer

```
print(math.floor(4.8))
```

4 — round down to nearest integer

```
print(math.pow(2, 10))
```

1024.0 — 2 to the power of 10

```
print(math.pi)
```

3.141592653589793

```
print(math.log(100, 10))
```

 2.0 — log base 10 of 100

TRY THIS LIVE

Ask the user to enter a loan amount and annual interest rate. Calculate the monthly EMI using `math.ceil()` to round up to the nearest rupee. Print: 'Your monthly EMI is Rs [amount].' This is a real use of math in an Indian finance context.

◆ The random Module

```
import random

print(random.randint(1, 100))           Random integer between 1 and 100 (inclusive)
print(random.random())                 Random float between 0.0 and 1.0

courses = ['Python', 'Data Science', 'Full Stack', 'UI/UX']
print(random.choice(courses))           Pick one random element from the list

students = ['Ravi', 'Priya', 'Mehul', 'Anjali', 'Harsh']
random.shuffle(students)                Shuffle the list in place
print(students)                         New random order each run

otp = random.randint(100000, 999999)    Generate a 6-digit OTP
print(f'Your OTP is: {otp}')
```

 Familiar to every Indian mobile user

04 Creating a Custom Module and Package

Any `.py` file you create is a module. Save your functions or classes in a file, then import that file from any other Python file in the same folder. This is how professional Python codebases are organized — one module per feature or responsibility.

```
# File: fee_calculator.py (your custom module)
Create this file

def calculate_emi(principal, rate, months):    Function in the module
    monthly_rate = rate / (12 * 100)
    emi = (principal * monthly_rate * (1 + monthly_rate)**months)
    emi /= ((1 + monthly_rate)**months - 1)
    return round(emi, 2)

GST_RATE = 18                                Module-level constant
```

```
# File: main.py (your main script)
import fee_calculator
emi = fee_calculator.calculate_emi(50000, 12, 12)
print(f'Monthly EMI: Rs {emi}')
print(f'GST Rate: {fee_calculator.GST_RATE}%')
```

Call the function

Access module constant

Create this in the same folder

Import the module by filename (no .py)

Uses the module's function

CONCEPT — Packages and `__init__.py`

A package is a folder that contains multiple related modules. To tell Python that a folder is a package (not just a directory), place an empty file called `__init__.py` inside it. You then import from the package using dot notation: `from mypackage.calculator import add`. This is how all major Python libraries like NumPy and Django are structured internally.

```
# Package folder structure:
# institute/
#   __init__.py
#   students.py
#   fees.py
#   reports.py

# Importing from the package:
from institute import students
from institute.fees import calculate_emi
import institute.reports as rep
```

Create this folder layout

Package folder

Empty file — marks this as a package

Module for student-related code

Module for fee-related code

Module for report generation

In your main.py file

Import the students module

Import one function from fees

Import reports module with alias

CASE STUDY — Reusable Utility Module at a Training Institute

Context — A Python development team at TOPS Technologies building an internal student management system across three different sub-applications: admissions, fees, and placements.

Challenge — The same helper functions — formatting names, calculating fee totals, validating phone numbers — were copy-pasted into all three sub-application files. A bug fix in one copy never made it to the others.

What they did — All shared functions were moved into a single `utils.py` module placed inside an `institute` package. All three sub-applications imported from it. The phone validator was fixed once and the fix propagated everywhere instantly.

Result — Code duplication dropped significantly. New team members only had to learn one set of shared utilities, not three different versions of the same logic.

Takeaway — A well-named custom module is the simplest form of DRY (Don't Repeat Yourself) — one

definition, used everywhere.

Source: Internal development case — TOPS Technologies, Ahmedabad, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the Modules and Standard Library lecture week.

Link — <https://nptel.ac.in/courses/106106145>

“A module is a promise: this file does one thing well, and anyone who imports it can trust that promise.”

WHERE TO GO NEXT

You can now import modules four ways, use math and random confidently, and build your own reusable modules and packages.

Explore deeper — Python Standard Library index — <https://docs.python.org/3/library/index.html>

Try on your own — Build an institute package with two modules: students.py (class Student with display method) and fees.py (function calculate_total_fee). Import both into a main.py and create two student objects, then calculate their fees.

Connect the dots — Session 09 introduces the re module — Python's Regular Expression library. You will use the import skills from today to load re and write patterns that validate and extract text from real-world strings.

Going further — Look up Python's if `__name__ == '__main__'` pattern. It is the professional way to write a file that works correctly both as a runnable script and as an importable module — essential for any real-world Python project.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain the difference between a module and a package in Python using a simple Hinglish analogy — like different departments in a company or different shelves in a library. Use an Indian example. Keep it under 150 words.

SESSION

09

60 MIN · TEXT PROCESSING

Regex — The re Module

◆ Learning Objectives

- Explain what a regular expression is and what problems it solves
- Identify and use the most common regex metacharacters and special sequences
- Use `re.match()` to check whether a string starts with a given pattern
- Use `re.search()` to find a pattern anywhere within a string
- Use `re.findall()` to extract all occurrences of a pattern from a string
- Write practical validation patterns for Indian phone numbers, email addresses, and pincodes

01 What Is a Regular Expression?

A regular expression (regex) is a sequence of characters that describes a pattern. You use it to check whether a string matches a certain format, to find specific text inside a larger string, or to extract and replace parts of it — all without writing long chains of if-else or string methods.

Every time a website in India tells you 'Enter a valid 10-digit mobile number' or 'Invalid email format', a regex is running behind the scenes. Python's `re` module brings that same power to your programs in just a few lines.

CONCEPT — Raw Strings in Regex

Regex patterns are almost always written as raw strings in Python: `r'\d+'` instead of `'\d+'`. A raw string tells Python to treat every backslash as a literal backslash — not as an escape character. Without `r'...'`, the pattern `'\d'` would require writing `'\\d'` to mean a single backslash. Use raw strings for every regex pattern to avoid confusion.

02 Metacharacters and Special Sequences

Regular expressions use a small set of special characters called metacharacters to describe patterns. A dot matches any character. Quantifiers like `*` and `+` control how many times something repeats. Anchors like `^` and `$` pin the pattern to the start or end of the string.

Symbol	Meaning	Example Match
.	Any single character except newline	<code>c.t</code> matches <code>cat</code> , <code>cut</code> , <code>cot</code>
<code>\d</code>	Any digit (0-9)	<code>\d\d\d</code> matches <code>123</code> , <code>456</code>
<code>\w</code>	Any word character (a-z, A-Z, 0-9, _)	<code>\w+</code> matches <code>hello</code> , <code>Ravi_2</code>

Symbol	Meaning	Example Match
\s	Any whitespace (space, tab, newline)	a\s b matches a b
^	Start of the string	^Hello matches Hello World
\$	End of the string	done\$ matches all done
*	Zero or more of the preceding	go* matches g, go, goo
+	One or more of the preceding	go+ matches go, goo (not g)
?	Zero or one of the preceding	colou?r matches color, colour
{n}	Exactly n repetitions	\d{6} matches 380001
{n,m}	Between n and m repetitions	\d{4,6} matches 3800, 38001
[abc]	Any one character from the set	[aeiou] matches any vowel
[^abc]	Any character NOT in the set	[^0-9] matches any non-digit

TRY THIS LIVE

Open a Python shell and import re. Try: `re.findall(r'\d+', 'My roll number is 1023 and PIN is 380001')`. The result should be `['1023', '380001']`. Change the pattern to `r'\d{6}'` and confirm only the 6-digit PIN is returned.

03 match(), search(), and findall()

The re module's three main functions each serve a different purpose. `match()` checks only at the start of a string. `search()` scans the whole string for the first occurrence. `findall()` returns every occurrence as a list. Each returns None (or an empty list) when no match is found — never an error.

```
import re
```

```
# re.match() – checks at the BEGINNING only
result = re.match(r'Hello', 'Hello World')
print(result.group())
```

Pattern is at the start — match found
Hello

```
result2 = re.match(r'World', 'Hello World')
print(result2)
```

Pattern is NOT at the start
None — match() does not scan forward

```
# re.search() – scans the ENTIRE string
result3 = re.search(r'World', 'Hello World')
```

Finds it anywhere in the string

```
print(result3.group())
```

World

```
# re.findall() — returns ALL matches as a list
text = 'Call 9876543210 or 8123456789 for info'
```

Two phone numbers in one string

```
phones = re.findall(r'[6-9]\d{9}', text)
print(phones)
```

Find all 10-digit Indian mobile numbers
['9876543210', '8123456789']

match() vs search()

match() only checks whether the pattern appears right at the beginning of the string. If the pattern is anywhere else, it returns None. Use match() for validating that a string starts with a specific format.

search() scans the entire string from left to right and returns the first match it finds anywhere. Use search() when you want to locate a pattern inside a larger block of text.

findall() vs search()

search() returns a Match object for the first occurrence only. To get the matched text you call .group() on it.

findall() returns a plain list of all matched strings. No .group() call needed. Use findall() when there can be multiple matches in the string — phone numbers, email addresses, dates, amounts. If there are zero matches, it returns an empty list (not None).

PRO TIP — Check for None Before Calling .group()

match() and search() return None when no match is found. Calling .group() on None raises an AttributeError and crashes your program. Always check first: 'if result: print(result.group())' — or use a conditional assignment. findall() is safer for this reason: it always returns a list, even an empty one.

04 Practical Patterns — Indian Validation Examples

Regex becomes most useful when you validate real-world data formats. Indian applications deal with specific formats: 10-digit mobile numbers that start with 6, 7, 8, or 9; 6-digit pincodes; email addresses; and Aadhaar numbers in groups of four digits. These patterns come up in nearly every form, API, and database validator you will build.

```
import re
```

```
# Indian mobile number — 10 digits, starts with 6-9
```

```
def is_valid_mobile(number):
```

```
    pattern = r'^[6-9]\d{9}$'
```

^ start, [6-9] first digit, 9 more digits, \$ end

```
    return bool(re.match(pattern, number))
```

Returns True or False

```
print(is_valid_mobile('9876543210'))
```

True

```
print(is_valid_mobile('1234567890'))
```

False — starts with 1

```
print(is_valid_mobile('98765'))
```

False — too short


```
# Indian pincode — exactly 6 digits
def is_valid_pincode(pin):
    return bool(re.match(r'^\d{6}$', pin))

print(is_valid_pincode('380001'))
print(is_valid_pincode('38000'))

# Basic email validation
def is_valid_email(email):
    pattern = r'^[\w.+-]+@[ \w-]+\.[\w.-]+$'
    return bool(re.match(pattern, email))

print(is_valid_email('ravi@tops.com'))
print(is_valid_email('ravi@tops'))
print(is_valid_email('ravitops.com'))
```

Exactly 6 digits, nothing else

True — Ahmedabad pincode

False — only 5 digits

username @ domain . extension

True

False — no extension

False — no @

CONCEPT — `re.compile()` for Reusable Patterns

If you use the same pattern many times, compile it once with `re.compile(pattern)`. This pre-processes the pattern and stores it as a reusable object, making repeated matching faster. Example: `mobile_re = re.compile(r'^[6-9]\d{9}$')`. Then call `mobile_re.match('9876543210')` without passing the pattern string every time.

CASE STUDY — Bulk Student Data Validation

Context — An admission office at a college in Gandhinagar importing 2,000 student records from an Excel file into their Python-based enrollment system.

Challenge — Roughly 15% of the records had invalid mobile numbers (7 or 11 digits, or starting with 1-5) and invalid pincodes. Importing bad data was corrupting the SMS notification system downstream.

What they did — A validation script using `re.match()` was written to check each mobile and pincode field before import. Invalid rows were written to a separate error CSV with the student name and the specific field that failed. Clean rows were inserted into the database.

Result — 320 invalid records were caught before import. The error CSV was sent back to the admissions team for correction in one afternoon. Zero bad records entered the system.

Takeaway — A handful of regex patterns can validate thousands of records in seconds — a task that would take days to do manually.

Source: Internal data migration case — TOPS Technologies partner college, Gandhinagar, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the String processing and regular expressions lecture.

Link — <https://nptel.ac.in/courses/106106145>

“A regex pattern is a contract: this string must look exactly like this — or it does not belong in my system.”

WHERE TO GO NEXT

You can now write regex patterns for `match()`, `search()`, and `findall()`, and validate real Indian data formats with confidence.

Explore deeper — Python official docs on the `re` module — <https://docs.python.org/3/library/re.html>

Try on your own — Build a form validator that asks the user for their name, mobile number, email, and pincode. Use four separate regex patterns to validate each field. Print 'Valid' or 'Invalid' for each, and only print 'Form submitted successfully' if all four pass.

Connect the dots — Session 10 moves into GUI programming with Tkinter — you will build a real desktop window with labels, input fields, and buttons, and you can wire the regex validators from today into the form submission logic.

Going further — Look up regex groups using parentheses — for example, `r'(\d{4})-(\d{2})-(\d{2})'` to extract the year, month, and day from a date string separately using `result.group(1)`, `result.group(2)`, `result.group(3)`.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain what a regular expression is and the difference between `re.match()`, `re.search()`, and `re.findall()` in simple Hinglish. Use an Indian example like validating a mobile number or an Aadhaar number format. Keep it under 200 words.

SESSION

10

60 MIN · DESKTOP APPLICATIONS

GUI Programming with Tkinter

◆ Learning Objectives

- Explain what Tkinter is and why it is the fastest way to build a Python desktop app
- Create a root window with a title, size, and event loop using Tk() and mainloop()
- Add Label, Button, and Entry widgets to a window and configure their properties
- Attach a Python function to a button using the command parameter
- Read user input from an Entry widget using .get() and display a result using .config()
- Arrange widgets in a window using the pack() and grid() layout managers

01 What Is Tkinter?

Tkinter is Python's built-in GUI (Graphical User Interface) library. It comes installed with Python — no pip install needed. With Tkinter you can build real desktop windows with buttons, input fields, labels, and menus in minutes, using only Python code.

A local shop in Rajkot or an institute in Ahmedabad often needs a simple desktop tool — a fee calculator, a student form, a daily report generator — that staff can use without opening a browser or a terminal. Tkinter is the quickest way to deliver that.

CONCEPT — Event-Driven Programming

A Tkinter application works differently from a script. A script runs top to bottom and exits. A Tkinter app creates a window, then enters an event loop — `mainloop()` — where it waits for the user to interact: click a button, type in a field, close the window. Each user action triggers a Python function called an event handler. `mainloop()` must always be the last line of your Tkinter code.

<code>import tkinter as tk</code>	Import Tkinter with alias tk
<code>root = tk.Tk()</code>	Create the main application window
<code>root.title('TOPS Student Portal')</code>	Set the window title bar text
<code>root.geometry('400x300')</code>	Width x Height in pixels
<code>root.resizable(False, False)</code>	Prevent resizing (optional)
<code>root.mainloop()</code>	Start the event loop — ALWAYS last

TRY THIS LIVE

Run the code above. A blank window should appear with the title 'TOPS Student Portal'. Try changing

geometry to '600x400' and run again. Then remove `root.resizable(False, False)` and drag the window corner to resize it.

02 Labels and Buttons

A Label displays static text or an image. A Button triggers a function when clicked. Both are created by passing the parent window as the first argument, followed by configuration options like text, font, and fg (foreground colour). Widgets are invisible until you call a layout method on them.

```
import tkinter as tk
from tkinter import messagebox

def on_click():
    messagebox.showinfo('Hello', 'Welcome to TOPS!')

root = tk.Tk()
root.title('My First App')
root.geometry('350x200')

# Label widget
lbl = tk.Label(root, text='Hello from Python!', font=('Arial', 16))
lbl.pack(pady=20)

# Button widget
btn = tk.Button(root, text='Click Me', command=on_click, bg='blue', fg='white')
btn.pack(pady=10)

root.mainloop()
```

Import messagebox for popups

Function called when button is clicked

Show a popup dialog

Create a label

pack() places widget — pady adds vertical space

command= wires the function

Place the button

CONCEPT — command= Must Be a Function Reference

The command parameter takes a function reference — the function name without parentheses. Writing `command=on_click` passes the function itself. Writing `command=on_click()` calls the function immediately when the window builds (before any click happens) and passes its return value, which is almost certainly wrong. Remember: no parentheses on the command.

03 Getting User Input with Entry

The Entry widget is a single-line text input field. The user types into it and your code reads the value using `.get()`. To update a Label's text after creation — for example to show a result — use `.config(text='new text')` on the label object.

```
import tkinter as tk

def greet():
    name = entry_name.get()
    if name.strip():
        result_lbl.config(text=f'Namaste, {name}!')
    else:
        result_lbl.config(text='Please enter your name.')

root = tk.Tk()
root.title('Greeting App')
root.geometry('350x180')

tk.Label(root, text='Enter your name:').pack(pady=8)
entry_name = tk.Entry(root, width=30)
tk.Button(root, text='Greet', command=greet).pack(pady=8)
result_lbl = tk.Label(root, text='', fg='green', font=('Arial', 12))

root.mainloop()
```

Read input and update result label
`.get()` returns the current text in the Entry
 Check the name is not empty
 Update the label text

Entry widget — stores a reference

Label above the input

Result label — empty at start

TRY THIS LIVE

Run the greeting app. Type your name and click Greet. Then clear the field and click Greet again — the else branch should fire. Try changing the result label colour from 'green' to 'red' for the empty-name error path.

04 Layout Managers and a Complete Mini App

Tkinter has three layout managers. `pack()` stacks widgets in a single direction — simple but limited. `grid()` places widgets in rows and columns — ideal for forms. `place()` positions widgets at exact pixel coordinates — precise but hard to maintain. For most applications, `grid()` is the professional choice.

Manager

How It Works

Best For

Manager	How It Works	Best For
<code>pack()</code>	Stacks widgets top-to-bottom or left-to-right	Simple one-column layouts, quick prototypes
<code>grid()</code>	Places widgets in a row/column table	Forms, calculators, any multi-column layout
<code>place()</code>	Absolute pixel position (x, y)	Splash screens, custom-positioned elements

The following EMI calculator brings together everything from this session — Entry for input, Button with command, Label for output, `grid()` for layout, and exception handling to catch non-numeric input. It also uses the `math` module from Session 08.

```
import tkinter as tk
from tkinter import messagebox
import math

def calculate_emi():
    try:
        principal = float(entry_amt.get())
        rate = float(entry_rate.get())
        months = int(entry_months.get())
        r = rate / (12 * 100)
        emi = (principal * r * (1 + r) ** months) / ((1 + r) ** months - 1)
        emi = math.ceil(emi)
        result_lbl.config(text=f'Monthly EMI: Rs {emi:,}')
    except ValueError:
        messagebox.showerror('Error', 'Please enter valid numbers.')
    except ZeroDivisionError:
        messagebox.showerror('Error', 'Rate or months cannot be zero.')

root = tk.Tk()
root.title('EMI Calculator')
root.geometry('360x240')

fields = [('Loan Amount (Rs):', 0), ('Annual Rate (%):', 1), ('Months:', 2)]
entries = []
for label_text, row in fields:
    tk.Label(root, text=label_text).grid(row=row, column=0, padx=12, pady=8, sticky='e')
    e = tk.Entry(root, width=20)
```

```
e.grid(row=row, column=1, padx=12, pady=8)
    entries.append(e)

entry_amt, entry_rate, entry_months = entries    Unpack for named access

tk.Button(root, text='Calculate EMI', command=calculate_emi, bg='#2563EB',
fg='white').grid(row=3, columnspan=2, pady=10)
result_lbl = tk.Label(root, text='', font=('Arial', 13, 'bold'), fg='#16a34a')
    Result label — updated on click
result_lbl.grid(row=4, columnspan=2)

root.mainloop()
```

PRO TIP — Never Mix pack() and grid() in the Same Window

Using both pack() and grid() on widgets inside the same parent window will cause Tkinter to throw a TclError and the app will freeze or crash. Pick one layout manager per container and stick with it. If you need different layouts in different sections, use Frame widgets as sub-containers — each Frame can use its own layout manager independently.

CASE STUDY — Desktop Fee Receipt Generator

Context — An accounts office at a private institute in Surat that collected fees across three batches and printed handwritten receipts.

Challenge — Staff were manually writing receipts and calculating totals by hand. Errors in arithmetic were frequent, and there was no digital record of daily collections.

What they did — A Python batch student built a Tkinter app in one week: three Entry fields for student name, course, and amount, a Calculate button that applied GST and generated a receipt number, and a Print button that exported the details to a text file.

Result — Receipt errors dropped to zero. The text file log became the institute's daily collection record, later imported into Excel for monthly reporting.

Takeaway — A Tkinter app built in days can replace a paper-based workflow that has existed for years.

Source: Student capstone project — TOPS Technologies, Surat batch, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the GUI and Tkinter programming week.

Link — <https://nptel.ac.in/courses/106106145>

“A window changes everything. The moment non-technical staff can click a button instead of run a script, your Python code becomes a real product.”

WHERE TO GO NEXT

You can now build a functional desktop application with input fields, buttons, result labels, and a proper grid layout.

Explore deeper — Tkinter official docs — <https://docs.python.org/3/library/tkinter.html>

Try on your own — Extend the EMI calculator: add a Clear button that resets all Entry fields and the result label. Add a second output line that shows the total amount repaid ($\text{EMI} \times \text{months}$) and the total interest paid (total repaid principal).

Connect the dots — Session 11 completes Module 3 by connecting Python to a real MySQL database — you will store, retrieve, update, and delete data from Python code, laying the groundwork for everything Django's ORM does automatically in Module 4.

Going further — Look up ttk (themed Tkinter widgets) — available as `from tkinter import ttk`. It provides Combobox, Treeview (for tables), and Notebook (for tabs), giving your app a modern look without any extra library.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. Explain how Tkinter works — what `mainloop()` does, why `command=` does not use parentheses, and the difference between `pack()` and `grid()` — in simple Hinglish. Use the analogy of a form at a government office or a bank branch. Keep it under 200 words.

SESSION

11

60 MIN · DATABASE

Database Connectivity with Python

◆ Learning Objectives

- Explain why Python programs need database connectivity beyond file storage
- Install PyMySQL and establish a connection to a local MySQL server
- Create a database table from Python using `cursor.execute()`
- Insert rows safely using parameterized queries to prevent SQL injection
- Retrieve rows using `fetchall()` and `fetchone()`, and display the results
- Update and delete records, and close the connection correctly using `close()`

01 Why Connect Python to a Database?

Text files work for small, simple data. But when your data grows — hundreds of students, thousands of transactions, multiple users writing at the same time — a relational database like MySQL becomes essential. It handles concurrent access, enforces data types, and lets you query exactly the records you need in milliseconds.

The Tkinter desktop app from Session 10 is the front end. MySQL is the back end where data actually lives. PyMySQL is the bridge — a Python library that lets your code talk to MySQL just like any other function call.

CONCEPT — PyMySQL vs `mysql-connector-python`

Two popular Python libraries connect to MySQL: PyMySQL (pure Python, easier to install on any OS) and `mysql-connector-python` (official Oracle driver, more features). Both use the same DB-API 2.0 interface — the same `connect()`, `cursor()`, `execute()`, and `close()` calls. This session uses PyMySQL. Switching to `mysql-connector` later requires almost no code changes.

```
# Step 1 – Install PyMySQL (run this in your terminal)
pip install PyMySQL                                Installs the library from PyPI

# Step 2 – Import and verify
import pymysql                                     Import the library
print(pymysql.__version__)                         Confirm the version installed
```

02 Connecting to MySQL

`pymysql.connect()` opens a connection to your MySQL server. You pass the host, username, password, and the database you want to use. Once connected, you create a cursor — an object that sends SQL statements to the database and retrieves results. Always close both the cursor and the connection when you are done.

```
import pymysql

# Open a connection to MySQL
connection = pymysql.connect(
    host='localhost',           MySQL server address (local machine)
    user='root',               MySQL username
    password='root123',        MySQL password
    database='institute_db',    Database to connect to
    charset='utf8mb4'          Support for all Unicode characters
)

cursor = connection.cursor()   Create a cursor to send SQL commands
print('Connected successfully!') Confirm connection

# Always close when done
cursor.close()                 Close the cursor
connection.close()             Close the connection — frees server resources
```

WARNING — Never Hardcode Passwords in Real Projects

Writing your MySQL password directly in the source file is only acceptable in learning environments. In any real project, store credentials in environment variables or a .env file and read them with `os.environ.get('DB_PASSWORD')`. If your code is ever pushed to GitHub with a hardcoded password, that credential is permanently exposed — even after deletion.

TRY THIS LIVE

Open MySQL Workbench and create a database called `institute_db`. Then run the connection code above. If you see 'Connected successfully!', your Python and MySQL are talking. If you get an error, check your MySQL service is running and your password is correct.

03 Creating a Table and Inserting Data

`cursor.execute()` sends any SQL statement to MySQL. For `CREATE TABLE`, you only need to call it — no commit required. For `INSERT`, `UPDATE`, and `DELETE` (any statement that changes data), you must call `connection.commit()` afterwards, or the change will not be saved permanently.

```
import pymysql

connection = pymysql.connect(host='localhost', user='root',
                             password='root123', database='institute_db')
```

```

cursor = connection.cursor()

# CREATE TABLE – create only if it does not already exist
cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        course VARCHAR(100),
        mobile VARCHAR(15),
        city VARCHAR(50)
    )
''')
connection.commit()
print('Table ready.')

```

Triple quotes for multi-line SQL

Save DDL changes

CONCEPT — Parameterized Queries (Always Use These)

Never build SQL strings by joining user input directly: 'INSERT INTO students VALUES (' + name + ')' is dangerous. If name contains SQL code, it can corrupt or destroy your database — this is called SQL injection. Parameterized queries pass values separately using %s placeholders. PyMySQL escapes them safely before sending to MySQL. Always use %s, never string concatenation, for user-supplied values.

```

# INSERT a single row – parameterized query
sql = 'INSERT INTO students (name, course, mobile, city) VALUES (%s, %s, %s, %s)'
values = ('Ravi Shah', 'Python', '9876543210', 'Ahmedabad')
cursor.execute(sql, values)
connection.commit()
print(f'Inserted row ID: {cursor.lastrowid}')

```

Safe, no SQL injection risk

%s = placeholder

Values passed separately

PyMySQL inserts values safely

REQUIRED — saves the insert permanently

Get the auto-generated ID

```

# INSERT multiple rows at once
batch = [
    ('Priya Patel', 'Data Science', '8765432109', 'Surat'),
    ('Mehul Desai', 'Full Stack', '7654321098', 'Vadodara'),
    ('Anjali Joshi', 'UI/UX', '9123456780', 'Rajkot')
]
cursor.executemany(sql, batch)
connection.commit()
print(f'{cursor.rowcount} rows inserted.')

```

executemany() for batch inserts

Insert all rows in one call

Confirm how many rows were added

04 Reading, Updating, and Deleting Data

After executing a SELECT query, you retrieve results using fetchall() for all rows, fetchone() for just the next row, or fetchmany(n) for a batch. Each row comes back as a Python tuple. For UPDATE and DELETE, always call commit() to persist the change.

```
# SELECT — fetch all rows
cursor.execute('SELECT * FROM students')
rows = cursor.fetchall()
for row in rows:
    print(row)
```

Run the query
Returns a list of tuples
Each row is (id, name, course, mobile, city)
(1, 'Ravi Shah', 'Python', '9876543210', 'Ahmedabad')

```
# SELECT with a WHERE filter
cursor.execute('SELECT * FROM students WHERE city = %s', ('Surat',))
result = cursor.fetchone()
print(result)
```

Parameterized — always
Note the comma — value must be a tuple
Get first matching row only
One row tuple, or None if not found

```
# UPDATE — change a record
cursor.execute('UPDATE students SET city = %s WHERE name = %s', ('Gandhinagar', 'Ravi Shah'))
connection.commit()
print(f'Rows updated: {cursor.rowcount}')
```

Must commit to save the update

```
# DELETE — remove a record
cursor.execute('DELETE FROM students WHERE name = %s', ('Anjali Joshi',))
connection.commit()
print(f'Rows deleted: {cursor.rowcount}')
```

Must commit to save the delete

```
# Always close both
cursor.close()
connection.close()
```

Frees the MySQL connection slot

PRO TIP — Use a DictCursor for Named Column Access

By default, fetchall() returns tuples — you access columns by index: row[1] for name. Change the cursor type to DictCursor and every row comes back as a dictionary: row['name'], row['city']. More readable and safer when column order changes. Create it with: cursor = connection.cursor(pymysql.cursors.DictCursor)

fetchall()

Returns every matching row as a list of tuples in one call. Loads everything into RAM. Use for small result sets where you need all rows at once — generating a report, displaying a full table.

fetchone()

Returns exactly one row at a time as a tuple, or None if there are no more rows. Memory-efficient for large results. Use when you need just one record — finding a student by ID — or to iterate through results one at a time in a while loop.

CASE STUDY — Placement CRM at TOPS Technologies

Context — TOPS Technologies, Ahmedabad, managing placement records for 500+ active students across seven training centres in Gujarat.

Challenge — Placement coordinators were tracking student status, job applications, and interview outcomes in a shared Excel file. Simultaneous edits caused overwriting, and there was no audit trail for status changes.

What they did — A Python script using PyMySQL connected to a centralized MySQL database on an EC2 instance. Each coordinator ran the script from their machine; all reads and writes went to the same database. Status updates were logged with a timestamp and coordinator ID.

Result — Data conflicts dropped to zero. Management could query real-time placement status across all centres from a single MySQL table. The script later became the data layer for a Laravel-based CRM.

Takeaway — Direct Python-to-MySQL connectivity is the fastest path from a data problem to a working solution — before frameworks and APIs are even needed.

Source: Internal system — TOPS Technologies Placement CRM, Ahmedabad, 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — IIT Madras. Look for the database connectivity and file handling advanced lecture.

Link — <https://nptel.ac.in/courses/106106145>

“A Python script that talks to a database is no longer just a script — it is the foundation of a real application.”

WHERE TO GO NEXT

You can now connect Python to MySQL, execute all four CRUD operations safely, and retrieve results as tuples or dictionaries.

Explore deeper — PyMySQL official docs — <https://pymysql.readthedocs.io/en/latest/>

Try on your own — Build a Python student registration system: a loop that asks for name, course, mobile, and city, inserts each student into MySQL, and prints all registered students from the database at the end. Use DictCursor so columns are readable by name.

Connect the dots — Module 4 begins with Django — a Python web framework that wraps everything you did manually in this session (connect, execute, commit, close) inside an ORM. You will write Python class definitions and Django does the SQL automatically. Understanding raw PyMySQL makes the ORM completely transparent.

Going further — Look up Python's context manager pattern for database connections: 'with pymysql.connect(...) as connection:' — it handles commit and rollback automatically, and closes the connection even if an exception occurs mid-query.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python and MySQL tutor. Explain the difference between `cursor.execute()` and `connection.commit()` in PyMySQL, using a simple Hinglish analogy — like placing an order at a restaurant vs the waiter actually sending it to the kitchen. Explain why forgetting `commit()` means your data is lost. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 3

Interview Questions

◆ Advance Python Programming

Rehearse each answer out loud and aim to deliver it within 60 seconds. The hint is only a nudge — attempt the answer first, then compare with the model answer below.

1

What does the `open()` function do in Python file handling?

Think about how you access files before reading or writing data.

MODEL ANSWER

`open()` establishes a connection between your Python program and a file on disk and returns a file object you can use to read or write. You pass it the filename and a mode string — for example, `open('data.txt', 'r')`. Until you open a file, Python has no way to interact with its contents.

2

Explain the difference between the file modes 'r', 'w', and 'a' when opening a file in Python.

One is for reading, one for writing (overwriting), and one for appending.

MODEL ANSWER

'r' opens the file for reading only — it raises `FileNotFoundError` if the file does not exist. 'w' opens for writing and creates the file if it does not exist, but if the file already exists it overwrites all existing content. 'a' also creates the file if absent but appends new content to the end without disturbing what is already there.

3

How would you write the string 'Hello, World!' to a new text file named 'greeting.txt' in Python?

MODEL ANSWER

```
with open('greeting.txt', 'w') as f:  
    f.write('Hello, World!')
```

Using the `with` statement ensures the file is automatically closed after the block finishes, even if an error occurs. Mode 'w' creates the file if it does not exist and overwrites it if it does.

4

What is the purpose of the `tell()` method in file handling?

It relates to the position of the file pointer.

MODEL ANSWER

`tell()` returns the current position of the file pointer as a byte offset from the beginning of the file. After opening a file and reading some data, `tell()` lets you know exactly where you are in the file. This is useful for saving your position before doing a `seek` or for debugging reading logic.

5

How can you move the file pointer to the beginning of a file after reading some lines?*There's a method that lets you set the cursor position.***MODEL ANSWER**

Call `seek(0)` on the file object to move the pointer back to the start. `seek()` takes a byte offset as its first argument and an optional reference point. `seek(0)` is the most common usage — it resets to the very beginning, allowing you to re-read the file without closing and reopening it.

6

Write a code snippet to read a file line-by-line and print each line.**MODEL ANSWER**

```
with open('data.txt', 'r') as f:
    for line in f:
        print(line.strip())
```

Iterating directly over the file object reads one line at a time without loading the entire file into memory — important for large files. `strip()` removes the trailing newline character that each line carries.

7

What is exception handling in Python, and why is it important?*Consider what happens when your code encounters errors at runtime.***MODEL ANSWER**

Exception handling lets your program respond to runtime errors gracefully instead of crashing. You wrap potentially failing code in a `try` block and write recovery logic in an `except` block. Without it, a single unexpected input — like a missing file or a non-numeric string — would terminate the entire program with an unhelpful traceback.

8

How do you handle multiple exceptions in a single try-except block?**MODEL ANSWER**

You can either list multiple `except` clauses, each catching a different exception type, or group them in a tuple in one clause. For example:

```
try:
    ...
except ValueError:
    print('Bad value')
except FileNotFoundError:
    print('File missing')
```

Or: `except (ValueError, TypeError) as e: print(e)` — catches both in one clause.

9

Write a try-except block that catches a ZeroDivisionError and prints 'Cannot divide by zero.'

MODEL ANSWER

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print('Cannot divide by zero.')
```

The try block attempts the division. When Python raises ZeroDivisionError, execution jumps to the except block instead of crashing. You can also add an else clause for code that runs only when no exception occurs.

10

How do you create a custom exception in Python?

It involves subclassing a built-in exception class.

MODEL ANSWER

Define a new class that inherits from Exception (or any built-in exception): class InsufficientFundsError(Exception): pass. You can add an __init__ method to include a custom message. Raise it with raise InsufficientFundsError('Balance too low'). Custom exceptions make error handling more expressive and self-documenting in larger applications.

11

What does the 'raise' keyword do in Python exception handling?

MODEL ANSWER

raise manually triggers an exception, either a new one or a caught one. For example, raise ValueError('Age cannot be negative') stops execution and propagates the error up the call stack. You can also use a bare raise inside an except block to re-raise the caught exception after doing some logging or cleanup.

12

If you see a long traceback after running your code, how can ChatGPT help you understand and fix the error?

Think about how you would use an AI tool to interpret error messages.

MODEL ANSWER

Copy the full traceback and paste it into ChatGPT with a brief description of what you were trying to do. ChatGPT can explain what the error type means, identify the line causing the problem, and suggest a fix. Always read the suggested fix carefully, apply it in your own code, and test again — do not paste AI code blindly, as the context of your full program matters.

13

What is a class in Python, and how is it different from an object?

One is a blueprint, the other is a specific instance.

MODEL ANSWER

A class is a blueprint or template that defines the attributes and behaviours a certain type of object will have. An object is a specific instance created from that blueprint. For example, Car is a class defining that all cars have a brand and speed; my_car = Car('Toyota', 120) is an actual object — a single car with those specific values.

14

Explain the purpose of the `__init__` method in a Python class.

MODEL ANSWER

`__init__` is the constructor method that Python calls automatically when you create a new object from a class. It initialises the object's attributes with the values passed in. For example, `def __init__(self, name): self.name = name` sets up the name attribute for each instance. Without it you would need to manually assign attributes after creation.

15

Write a simple Python class named 'Car' with attributes 'brand' and 'year'. Create an object of this class.

MODEL ANSWER

```
class Car:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

my_car = Car('Toyota', 2022)
print(my_car.brand, my_car.year) # Toyota 2022
```

`self` refers to the current instance. Each object created from Car can have its own brand and year values.

16

What is single inheritance in Python? Give an example scenario where you might use it.

MODEL ANSWER

Single inheritance is when a child class inherits attributes and methods from exactly one parent class. For example, Animal can be the parent with a `speak()` method, and Dog can inherit from Animal and override `speak()` to bark. This lets Dog reuse Animal's common logic while customising only what is different.

17

How does multiple inheritance differ from multilevel inheritance in Python?

Think about the number of parent classes and the inheritance chain.

MODEL ANSWER

Multiple inheritance means one child class inherits from two or more parent classes at the same level — `class C(A, B)`. Multilevel inheritance is a chain: class A is the parent of B, and B is the parent of C, forming a hierarchy. Multiple inheritance is useful when combining features from distinct classes; multilevel is for building specialised versions progressively.

18

Given a 'Grandparent', 'Parent', and 'Child' class, how would you implement multilevel inheritance?

MODEL ANSWER

```
class Grandparent:
    def greet(self):
        print('Hello from Grandparent')

class Parent(Grandparent):
    pass

class Child(Parent):
    pass

obj = Child()
obj.greet() # Hello from Grandparent
```

Child inherits from Parent, which inherits from Grandparent, so Child can access all methods defined anywhere in the chain.

19

What is polymorphism in Python? Explain with an example.

MODEL ANSWER

Polymorphism means 'many forms' — different classes can define the same method name, and Python calls the correct version based on the object type at runtime. For example, a Dog class and a Cat class both have a sound() method; dog.sound() barks and cat.sound() meows. Code that calls obj.sound() works for any animal object without needing to know the specific class.

20

Does Python support method overloading in the same way as languages like Java? Explain your answer.

Focus on how Python handles functions with the same name but different parameters.

MODEL ANSWER

No — Python does not support traditional method overloading. If you define two methods with the same name, the second one simply replaces the first. Instead, Python achieves similar behaviour using default parameter values and *args or **kwargs to accept a variable number of arguments. This gives one function the flexibility to handle multiple calling patterns.

21

How does method overriding work in Python? Give a code example.

MODEL ANSWER

```
class Animal:
    def speak(self):
        print('Some sound')

class Dog(Animal):
    def speak(self):
        print('Bark')

d = Dog()
d.speak() # Bark
```

When Dog defines speak(), it overrides the parent's version. Calling speak() on a Dog object always uses Dog's definition. Use super().speak() inside the child to also call the parent's version if needed.

22

What is the purpose of a module in Python?

Think about code organisation and reuse.

MODEL ANSWER

A module is a single .py file that groups related functions, classes, and variables together. It keeps code organised, prevents name collisions, and lets you reuse logic across multiple projects simply by importing it. Python's standard library is itself a large collection of modules — math, os, datetime, and so on.

23

How do you import a built-in module, such as 'math', and use its 'sqrt' function?

MODEL ANSWER

```
import math
result = math.sqrt(25)
print(result) # 5.0
```

Alternatively, from math import sqrt; result = sqrt(25) imports only the function you need. The first approach is safer in larger projects to avoid name collisions; the second is convenient for one-off use.

24

Write the steps to create your own Python module and import it into another script.

MODEL ANSWER

Create a file called utils.py and define a function: def add(a, b): return a + b. In a second file (main.py) in the same directory, write import utils and call utils.add(3, 4). Both files must be in the same folder, or the module's folder must be in the Python path. That is all — any .py file is a module.

25

What is the difference between a module and a package in Python?

MODEL ANSWER

A module is a single .py file containing Python code. A package is a directory that contains multiple modules along with an `__init__.py` file, allowing the modules to be imported as a group. For example, `mypackage/student.py` is a module, while `mypackage/` itself is the package. Packages let you organise related modules under a common namespace.

26

What is the use of the 're' module in Python?

It's related to working with patterns in strings.

MODEL ANSWER

The re module provides functions for working with regular expressions — patterns used to search, match, or manipulate strings. Common uses include validating email addresses or phone numbers, extracting data from text, and find-and-replace operations. Key functions include `re.match()`, `re.search()`, `re.findall()`, and `re.sub()`.

27

How does the `match()` function in the re module differ from `search()`?

MODEL ANSWER

`re.match()` checks for a pattern only at the very beginning of the string — if the pattern does not start at position 0, it returns `None`. `re.search()` scans through the entire string and returns the first location where the pattern is found. Use `match()` when the pattern must be at the start, and `search()` when it can appear anywhere.

28

Write a Python code snippet to find all occurrences of the word 'python' in a text using `re.findall()`.

MODEL ANSWER

```
import re
text = 'I love Python. Python is powerful. python rocks.'
matches = re.findall(r'python', text, re.IGNORECASE)
print(matches) # ['Python', 'Python', 'python']
```

`findall()` returns a list of all non-overlapping matches. The `re.IGNORECASE` flag makes the search case-insensitive so 'Python', 'PYTHON', and 'python' are all found.

29

What is Tkinter used for in Python?

MODEL ANSWER

Tkinter is Python's built-in library for creating graphical user interface (GUI) applications. It provides widgets like buttons, labels, text fields, checkboxes, and windows that let you build desktop apps without installing any third-party libraries. It is cross-platform and ships with the standard Python installation.

30

How would you create a basic Tkinter window with a label that says 'Welcome'?

Think about the sequence: import, create window, add label, mainloop.

MODEL ANSWER

```
import tkinter as tk
root = tk.Tk()
root.title('My App')
label = tk.Label(root, text='Welcome')
label.pack()
root.mainloop()
```

mainloop() starts the GUI event loop and keeps the window open until the user closes it. Without it the window would appear and immediately disappear.

31

Describe how you would add a button to a Tkinter window that prints 'Button clicked' when pressed.

MODEL ANSWER

```
def on_click():
    print('Button clicked')

btn = tk.Button(root, text='Click Me', command=on_click)
btn.pack()
```

The command parameter links the button to a function. When the user clicks the button, Tkinter calls on_click(). The function can update labels, open dialogs, or trigger any logic.

32

What are the common layout managers in Tkinter, and how do they differ?

Focus on pack, grid, and place.

MODEL ANSWER

pack() stacks widgets automatically in order — top to bottom or left to right — and is quick for simple layouts. grid() organises widgets in a row-column table structure, giving you precise control over alignment and is better for forms. place() positions widgets at exact pixel coordinates, offering the most control but requiring manual adjustment when the window resizes.

33

How do you connect a Python application to a MySQL database using pymysql?

Think about the connect function and required parameters.

MODEL ANSWER

```
import pymysql
conn = pymysql.connect(host='localhost', user='root', password='yourpassword', database='mydb')
cursor = conn.cursor()
```

The connect() function takes the host, username, password, and database name. The returned connection object lets you create a cursor for executing SQL queries. Always close the connection after your work is done.

34

Write a code snippet to insert a new record into a 'students' table using Python and pymysql.

MODEL ANSWER

```
sql = 'INSERT INTO students (name, age) VALUES (%s, %s)'
cursor.execute(sql, ('Ravi', 21))
conn.commit()
```

Using %s placeholders with a tuple of values is parameterised query syntax — it prevents SQL injection. conn.commit() is required to save the change to the database permanently.

35

Explain how you would fetch all rows from a table in MySQL using Python.

MODEL ANSWER

```
cursor.execute('SELECT * FROM students')
rows = cursor.fetchall()
for row in rows:
    print(row)
```

fetchall() returns all result rows as a list of tuples. If you expect a large result set, fetchone() retrieves one row at a time to save memory. Always execute a SELECT before calling fetch methods.

36

What precautions should you take when working with databases in Python to avoid SQL injection?

MODEL ANSWER

Always use parameterised queries with placeholders (%s in pymysql) rather than building SQL strings by concatenating user input. Validate and sanitise all user-supplied data before using it in a query. Limit database user permissions to only what the application needs, and never store database credentials in plain text inside your source code.

37

How would you use both regular expressions and file handling together to extract all email addresses from a text file?

Combine reading from a file and searching with a pattern.

MODEL ANSWER

```
import re
with open('contacts.txt', 'r') as f:
    content = f.read()
emails = re.findall(r'[w.-]+@[w.-]+\w+', content)
print(emails)
```

The file is read into a string, and findall() applies the email pattern across the entire content at once. This approach handles files of any size and returns a clean list of all matched addresses.

38

If you have a class that manages database connections and you want to ensure it only creates one connection object, which OOP concept would you use and how?

MODEL ANSWER

Use the Singleton design pattern. Override `__new__` to check if an instance already exists — if it does, return the existing one; if not, create it and store it as a class variable. This guarantees that no matter how many times you instantiate the class, only one database connection is ever created, preventing resource waste and connection pool exhaustion.

39

How could you use ChatGPT or GitHub Copilot to help you debug a Python script that is not working as expected, and what would you do to make sure the suggestions are safe to use?

Think about verifying AI-generated code and understanding its logic.

MODEL ANSWER

Share the problematic code and the error message or unexpected output with ChatGPT and ask it to explain what is wrong and how to fix it. Before applying any suggestion, read it to understand the change, check that it does not introduce hard-coded credentials or unsafe inputs, and run your tests again. For database or file-handling code, pay particular attention to whether the suggested fix maintains parameterised queries and proper resource cleanup.